```
# Anaconda Distribution
# Jupyter Notebbok
# Operating System
# Programmming Languages
# High Level and Low Level Langugaes
```

Translator

```
# - It is a small software that converts the HLL code into LLL code and vice-versa.
```

Types of Translators

```
# - 1- Compiler
# - 2- Interpreter
```

1- Compiler

```
# - It reads the whole source code (HLL code) first then it converts the whole source code to machine code at one go.
```

2- Interpreter

```
# - It reads the source code line by line and converts into machine code.
# - It reads and converts one line at a time.
```

Compiled and Interpreted Languages

```
# - Any language that uses compiler is a compiled language.
# Example - Java, C, C++ etc.

# - Any language that uses interpreter is an interpreted language.
# Example - Python, R, PHP, Ruby etc.
```

Python is compiled or Interpreted?

How a Python progra gets executed?

Source Code

```
# - The code that we write as a programmer is known as source code.
# - The file extension of the Python source code is .py.
```

Intermediate Code

```
# - When we run a Python program.
# - First compiler comes into  picture.
# - The compiler reads the whole source code first and if there is no error then it converts the source code into intermediate
# - If compiler catches any error in the source code then it does not convert it to intermediate code.
# - In this case we don't get any output.
# - The Python intermediate code is also known as bytecode.
# - The file extension of Python bytecode is .pyc.
```

Machine Code

```
# - As soon as Bytecode gets generated, interpreter comes into picture.
# - It reads the bytecode line by line and converts it into machine code.
# - If interpreter catches any error it stops the execution right away.
# - Until interpreter catches any error, we get the output.
```

Types of Errors

```
# - 1- Compiler Time Errors
# - 2- Run Time Errors
```

1- Compile Time Errors

```
# - The errors that are caught by the compiler are the compile time error.
```

```
# Examples-
# - 1- Syntax Error
# - 2- Indentation Error
```

```
a = 29
b = 49
if a>30:
    if b>50:
        print("B is greater than 50")
    else:
        print("B is less than 50")
else:
    print("A is not greater than 30")
```

```
A is not greater than 30
```

```
a = int(input("Enter a no : "))
b = int(input("Enter second no : "))
c = a+b
print(f"Addition of {a} and {b} is ", c)
```

```
Enter a no :  35
Enter second no :  25
Addition of 35 and 25 is  60
```

Run Time Errors

```
# - The errors that occur at run time (execution) are known as the run time errors.
# - Interpreter is responsible to catch the run time errors.
```

```
# Examples-

# - Name Error
# - Type Error
# - Value Error
# - Index Error
# - Key Error
# - Attribute Error
# - Zero Division Error
# - Stop Iteration Error
# etc.
```

```
a=30
b=25
print(b)
```

```
25
```

```
a=40
a=50
b=60
```

```
print(a)
print(b)
print(c)
print(a)
```

```
50
60
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[37], line 3
      1 print(a)
      2 print(b)
----> 3 print(c)
      4 print(a)

NameError: name 'c' is not defined
```

```
# code -1
# code -2
# code -3
# Syntax Error (compile time error)
# code -4
```

```
# # code -1
# # code -2
# # code -3
# # code -4 Till here we will get the output
# Name Error (Run time error)
# code -6
# code -7
```

```
# # code -1
# # code -2
# # code -3
# # code -4
# Name Error (Run time error)
# Indentation Error (compile time error)
# code -7
```

```
print('Hello friends!')
a=10
b=20
c=a+b
print(c)
print('Thanks')
```

```
Hello friends!
30
Thanks
```

```
print('Hello friends!')
a=10
b=20
c=a+b
print(c)
print('Thanks'
```

```
  Cell In[67], line 6
    print('Thanks'
                   ^
SyntaxError: incomplete input
```

```
print('Hello friends!')
a=10
b=20
c=a+b
print(c)
print(d) # Name Error
print('Thanks')
```

```
Hello friends!
30
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[69], line 6
      4 c=a+b
      5 print(c)
----> 6 print(d) # Name Error
      7 print('Thanks')

NameError: name 'd' is not defined
```

```
print('Hello friends!')
a=10
b=20
c=a+b
print(c)
print(d)
print'Thanks')
```

```
  Cell In[1], line 7
    print'Thanks')
                  ^
SyntaxError: unmatched ')'
```

Why Python?

```
# - Extensive Library support
# - Strong community
# - Continuous Development
# - Open source
```

Contents

```
# 1- Data Types
# 2- Typecasting
# 3- Input and Output Functions
# 4- Operators
# 5- String
# 6- Conditional Statements
# 7- Loops
# 8- List
# 9- Tuple
# 10- Dictionary, Set, Frozenset
# 11- Functions
# 12- Modules
# 13- File Handling
# 14- Exception Handling
# 15- Iterators and Generators
# 16- Regex
# 17- NumPy
# 18- Pandas
# 19- Matplotlib
# 20- Seaborn
```

Data Types

```
# - It is a classifier that classifies the type of a value (object)
```

```
# Different type of data -> Different memory requirement
# Different type of data -> Different type of operation
# It is important to know the type of the data.
```

Types

```
# - 1- Premetive Datatypes
# - 2- Derived Datatypes
```

1- Premetive Datatypes

Integer - int

```
# - Any whole number whether +ive, -ive or zero is an int value.

# type() is used to get the type of the data.
```

```
a=5
print(a)
print(type(a))
```

```
5
<class 'int'>
```

```
a=-2
print(a, type(a))
```

```
-2 <class 'int'>
```

```
a=0
print(a, type(a))
```

```
0 <class 'int'>
```

Float - float

```
# - Any number with a decimal point is a float value.
```

```
a=2.5
print(a, type(a))
```

```
2.5 <class 'float'>
```

```
a=-6.8
print(a, type(a))
```

```
-6.8 <class 'float'>
```

```
a=0.0
print(a, type(a))
```

```
0.0 <class 'float'>
```

Complex - complex

```
# - Any number in the form of a+bj is a complex value.
# - a is the real number whereas b is the imaginary number.
```

```
a=2+5j
print(a, type(a))
print(a.real)
print(a.imag)
```

```
(2+5j) <class 'complex'>
2.0
5.0
```

```
a=2.3+6.8j
print(a, type(a))
```

```
(2.3+6.8j) <class 'complex'>
```

```
a=-5+6j
print(a, type(a))
```

```
(-5+6j) <class 'complex'>
```

```
a=5+7j
print(a)
```

```
(5+7j)
```

```
a=5j
print(a, type(a))
```

```
5j <class 'complex'>
```

```
a=5+6j
print(abs(a))
```

```
7.810249675906654
```

```
print((5**2+6**2)**.5)
```

```
7.810249675906654
```

String - str

```
# - Anything inside the '', "", '''''', """""" is a string value.
```

```
a="55"
print(a, type(a))
```

```
55 <class 'str'>
```

```
a='Hello freinds!123!@#$%^^'
print(a, type(a))
```

```
Hello freinds!123!@#$%^^ <class 'str'>
```

```
a='Python'
print(a, type(a))
```

```
Python <class 'str'>
```

```
a="Python"
print(a, type(a))
```

```
Python <class 'str'>
```

```
# Output- Python's libraries
a="Python's libraries"
print(a, type(a))
```

```
Python's libraries <class 'str'>
```

```
# Output -> This is "Python" class

a='This is "Python" class.'
print(a, type(a))
```

```
This is "Python" class. <class 'str'>
```

```
a='''Hello friends!
How are you doing?'''

print(a, type(a))
```

```
Hello friends!
How are you doing? <class 'str'>
```

```
a="""Hello friends!
How "are" you doing?"""

print(a, type(a))
```

```
Hello friends!
How "are" you doing? <class 'str'>
```

```
#  Output -> 'Hello friends!
# How are you doing?'
```

```
a= """'Hello friends!
How are you doing?'"""

print(a, type(a))
```

```
'Hello friends!
How are you doing?' <class 'str'>
```

```
#  Output -> "Hello friends!
# How are you doing?"
```

```
a='''"Hello friends!
How are you doing?"'''

print(a, type(a))
```

```
"Hello friends!
How are you doing?" <class 'str'>
```

Boolean - bool

```
# True and False are the boolean values.
```

```
a=True
print(a, type(a))
```

```
True <class 'bool'>
```

```
a=False
print(a, type(a))
```

```
False <class 'bool'>
```

Hello

Hello my `friend`"

```
Start coding or generate with AI.
```